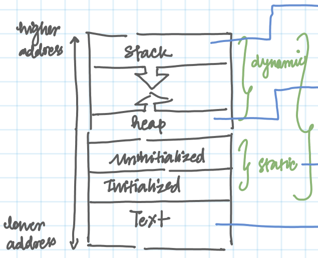
"To be insanely intelligent and fall in love with learning"
"Just Work Hard and keep growing"
"If it is your calling, it will keep you calling"

22nd July 2025 — problem → understand → flowchart → program

Foundations of C++

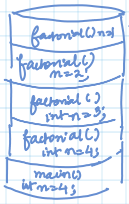
1) Data is stored in RAM



- Stack: function calls, local variables, input arguments, fifo
- Heap (dynamic): variable in size, you can decide its size using malloc, free - memory leaks! used for linked list, dynamic structure
- Data (static): initialized data segment - store var & is R/W → a → 6, 5, 7 ...
  uninitialized or BSS (block started by symbol) R/W set to 0. stored seperately from initialized as the 0 is implicitly set, not each var. array → 0,0,00 × ; array → 128, 15 (size)
- Text: Read only, contain code, fixed and read directly for execution (compiled code)

```cpp
int fact(int n)
{ if (n==0 || n==1)
    { return 1; }
    return n*factorial(n-1); }

int main()
{ int n=4;
  printf("%d", factorial(n)); }
```



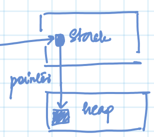
- Every function call creates a stack frame.
- Recursion calls multiply this.

type	size	
char	1	
bool	1	
short	2	
int	4	
long	4/8	
long long	8	
float	4	7 decimal precision
double	8	15 decimal
void*	8	pointer type

Ranges:

char	(−128, 127)
unsigned char	(0, 255)
int	(−2,147,483,648, 2,147,483,647) or $\pm 2*10^9$
unsigned int	(0, 4,294,967,295)
long long	$\pm 9*10^{18}$
float	$\pm 10^{38}$ (~6-7 digits)
double	$\pm 10^{308}$ (~15 digits)
void *	64 bit

How std containers work?



vector<int> v;

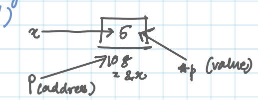
vector lives in stack but hold a pointer to heap allocated memory in stack

so v = {1,2,3} → heap; stack ---- pointer

Pointers & References

```cpp
int x=5;
int* p = &x;   // store address of x
cout << *p;
```

dereference x and gives 10 (actual data stored at address)



Functions: value vs Reference

```cpp
void f(int x) {x = 20;}
void fref(int& x) {x=20;}
```

- pass by value
- * pass by reference — to avoid copy & to modify in place

Arrays:

when you pass it to a function or perform operation it acts as a pointer.

```cpp
int arr[] = {9,8,7};
cout<< *(arr+1);   //output 8
```

arr → arr[0] (address)
arr+1 → arr[0+1] (add)
*(arr+1) → 8 (value)

Cache locality: array store in contiguous memory so CPU prefetch the next elements (& values) (say a chunk) into cache line. This results in faster access time and efficient iteration & traversal.

Avoid using list<T>, map<T>, set<T>, as they lead to memory lookups.
when iterating over large data use vector for cache friendly performance.

★ STL Container methods - Must know

vectors — Dynamic array, sliding window, push-back/pop-back

1.	push_back(val)	append to end	O(1)
2.	pop_back()	remove last element	O(1)
3.	size()	no of elements	O(1)
4.	empty()	check vector is empty	O(1)
5.	clear()	remove all elements	O(N)
6.	resize(n)	resize vectors to n elements	O(N)
7.	front() / back()	access first/last element	O(1)
8.	begin() / end()	return iterators	O(1)
9.	insert()	insert at any position	O(N) worst
10.	erase()	remove at any position/range	O(N) worst

always reverse() when size is predictable

Why not emplace_back instead of push_back?

eg: v.push_back(Person("A", 30));
v.emplace_back("A", 25);

- push_back will create Person → copy to vector
- emplace_back will directly add it to vector

Use emplace_back to avoid temp obj's in structs, trees, graphs etc

map<K, V> — ordered (red black tree)

insert({k,v})	insert or update k	O(log N)
erase(k)	remove key	O(log N)
find(k)	get iterator to key	O(log N)
operator[]	access or insert key	O(log N)
size()	number of pairs	O(1)
begin() / end()	ordered iteration	O(1)
lower_bound(k)	first key >= k	O(log N)
upper_bound(k)	first key > k	O(log N)

for frequency count, range queries, prefix sums, intervals

unordered_map<K,V>

insert key value	O(1) on avg, O(N) in worst	
remove key	O(1)	
find key	O(1)	
insert/access	O(1)	
return 1 if key exist	O(1)	
no of elements	O(1)	

Hashing, 2sum, frequency counting, caching, grouping values or indices

set<T> Ordered set (Balanced BST)

			unordered_set<T>
insert(val)	insert value	O(log N)	O(1)
erase(val)	remove val	O(log N)	O(1)
find(val)	check if val exist	O(log N)	O(1)
count(val)	check presence	O(log N)	O(1)
lower_bound(val)	first value ≥ val	O(log N)	
upper_bound(val)	first value > val	O(log N)	
begin() / end()	sorted iteration	O(1)	

unordered_set: size() → O(1)

set: unique sorted data, order statistics, min/max window problems

unordered_set: unique storage with no order, fast lookup, hashing, seen/unseen value tracking

Basic maths:

1) a%b → b)a(a' ; r → ans. Eg: 4%2 = 2)4(2 ; 4 ; 0 → ans, 4%2=0

2) %10 gives last digit
   /10 removes last digit

3) XOR → $a \wedge a = 0$, $a \wedge 0 = a$, $a \wedge b \wedge a = b$ — to find unique elements in arrays, swapping without temp, toggling bits

```cpp
int a=5, b=10
a = a^b;   -①
b = a^b;   (a^b)^b = a  -②
a = a^b;   (a^b)^a = b
```

4) to find power of 2 — O(1)
   n & (n-1) == 0

eg: n=8 → 1000
n-1=7 → 0111
1000 & 0111 = 0000 → 0
hence 8 is a power of 2

learn as you solve question. There are gazallion math concepts you might or might not need.

24.07.2025

Logic building question

1) Basic Problems

Q1 check even or odd

```
bool isEven(int n){
    if (n%2 == 0)
        return true;
    else
        return false;
```

odd numbers:

- 5 → 0 1 0 1
- 7 → 0 1 1 1
- 3 → 0 0 1 1
- 15 → 1 1 1 1

all have 1 as the last bit → 0101 & 1 = 0001 so it will always be 1

```
if (n & 1 == 1)
    return false;
else return true;
```

Q. Print multiplication table

n = 1, 2, ... O(11) or O(1)

```
for (int i=1; i<11; i++)
    cout << n << " * " << i << " = " << n*i << endl;
```

worst way:

```
void printTable(int n, int i=1){
    if (i == 11) return;
    cout << n << " * " << i << " = " << n*i << endl;
    i++;
    printTable(n, i);
}
```

Q. Sum of natural numbers

```
int sumOfNaturalNumbers(int n){
    if (n==0 || n==1) return n;
    int sum=0;
    for (i=1; i<=n; i++)
        sum += i;          O(n)
    return sum;
}
```

using formula → $n\frac{(n+1)}{2}$ 5+4+3+2+1 = 15   O(1)

eg: n=5 → $\frac{5(5+1)}{2} = \frac{5 \times 6}{2} = 15$

`return (n*(n+1))/2;`

using Recursion

```
int findSum(int n){
    if (n==1)
        return 1;          O(n)
    return n + findSum(n-1);
}
```

n=3: 3 + findSum(2) → 2 + findSum(1) → return 1; 2+1; 3 = 3+3 = 6

Q. Sum of squares of first n natural numbers

```
for (int i=1; i<=n; i++)
    sum += i*i;
```

O(n)

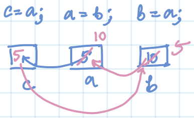
formula: $n(n+1)(2n+1)/6$

return `(n*(n+1)*(2n+1))/6`

Q. Swap 2 numbers:

Brute force:

```
int main(){
    int a=5; int b=10;
    int c=0;
    c=a;  a=b;  b=a;
```



memory = O(1)

optimal - no memory: swap(a, b)

a=5; b=10

```
a = a^b;
b = a^b;
a = a^b;
```

5^10 — ①
5^10 ^ 10 = 5 — ②
5^10 ^ 5 = 10

or

```
a = a+b;   15
b = a-b;   15-10 = 5
a = a-b;   15-5 = 10
```

Q. Closest to n & divisible by m

Brute force

n = 13, m = 5: 10 ÷ 5 ✓, 11, 12, 13, 14, 15 ÷ 5 ✓ closest!

- loop won't work as data is +ve & -ve
- 13%5 = 3   5*(2) & 5*(2+1)

n = -15, m = 2: 12, 13, 14, 15, 16, 17, 18 — either can be used!

- 2) -15 (-7   2*(-7) & 2*(7+1)

Given → m, n   To find → k such that m÷k and closer to n

lets find the possible k's

```
int closestNumber(int m, int n){
    int quotient = n/m;
    int k1 = m * quotient;
    int k2 = n*m > 0 ? m*(quotient+1) : m*(quotient-1);
    if (n-k1 < n-k2)
        return k1;
    else
        return k2;
```

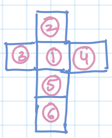
13/5 = 2; 5×2 = 10; 5×3 = 15

-18/5 = -3; k_1 = 5*-3 = -15; k_2 = 5*(-3+1) ✗ = -10; ✓ 5*(-4) = -20 by checking n*m > 0 as both can be < 0 too!

-17/-4: k_1 = -4*4 = -16; k_2 = -4*(4+1) = -4*5 = -20. -16 is the closest

Q. Dice problem

Given - cube dice with 6 faces
to find - opposite face of cube



- 2 → 5   5+2 = 7
- 1 → 6   6+1 = 7   magic!
- 3 → 4   4+3 = 7

```
int oppositeDiceFace(int n){
    return 7-n;
}
```

O(1).

Q. nth term of ap from 1st 2 terms

formula: $t_n = a_1 + (n-1)*d$ difference in common series $d = a_2 - a_1$

```
int nthTermOfAp(int a1, int a2, int n){
    return (a1 + (n-1) * (a2-a1));
}
```

$a_1 = 2,\ a_2 = 3,\ n = 4$

$2 + (4-1)*(3-2) = 2 + 3*1 = 5$

Q. Sum of digits of a number

```
int sumOfDigits(int n){
    int sum=0;
    while (n != 0)
    {
        sum += n%10;
        n = n/10;
    }
```

$O(\log_{10} n)$

recursion:

```
int sumOfDigits(int n)
{
    if n==0
        return 0;
    return (n%10) + sumOfDigits(n/10);
}
```

Q. Reverse of Digits

```
int reverseOfDigits(int n){
    reverse_no = 0;
    while (n != 0) {
        reverse_no = (n%10) + reverse_no * 10;
        n = n/10;
    }
```

687 = 700 + 80 + 6; (n%10) = 7; (n/10) = 68

n = 687: ① 7 ② 8+70 = 78 ③ 6+780 = 786

n%10 = 7, n%10 = 8, n% = 6; n*100, n*10, n*1

Another method for O(1) space

```
string s = to_string(n);
reverse(s.begin(), s.end());
n = stoi(s);
return n;
```

Q. Prime testing

Given → prime no   to find → prime or not, print true or false

no is prime if → it is divisible by 1 and itself

11 → (1, 11), 13 → (1, 13), 7 → (1, 7)

```
bool isPrime(int n);      O(n), O(1)
{
    for (int i=2; i<=n; i++)
    {   if (n%i == 0)
            return false; }
    return true;
}
```

$O(\sqrt{n})$ or i <= sqrt(n)

```
for (int i=2; i*i<=n; i++)
{ if (n%i == 0)
      return false; }
return true
```

as 36 → 1×36, 2×18, 3×12, 4×9, 6×6 | 9×4, 12×3, 18×2, 36×1

$\sqrt{36} = 6$ gives all the factors.

Q. Check power

Brute:

```
int pow_value = 0; i = 0;
while (pow_value < y)
{   pow_value = x^i;
    i++;
}
if (pow_value == y)
    return true;
else
    return false;
```

$O(\log_2 y)$ why?

as $1^n = 1$ so check if $y==1$ as only then it is true

2) Better approach

use binary search

```
if (x==1) return (y==1)
if (y==1) return true; // as x^0 = 1

long int pow=x, i=1;
while (pow<y) {
    pow *= pow;
    i *= 2;
}
if (pow==y) return true;
```

get the search range $\neq 2^n$

very hard take it

– binary search

```
while ( low<=high) {
    long int mid = low + (high-low)/2;
    long int result = pow( x, int ( log2(mid)/log2(x) ));
```

3) optimal

```
float res1 = log(y)/log(x);
return res1 == floor(res1);
```

$y = 1000$, $x = 10$

$\frac{\log(1000)}{\log(10)}$ = whole no.

```
bool isPower(int x, int y) {
    if (x==1) return y==1;
    double res = log(y)/log(x);
    return abs(res – round(res)) < 1e-9;
}
```

Logarithms:

$b^p = n \rightarrow \log_b n = p$

$2^3 = 8 \rightarrow \log_2 8 = 3$

Q. Distance b/w 2 points

euclidean distance:

```
float getDist(int x1, int x2, int y1, int y2) {
    return
    sqrt( ( (pow(abs(x2-x1), 2) +
            pow(abs(y2-y1), 2)) * 1.0);
```

can be skipped as sqrt is applied (-ve)*(-ve) = +ve

or any number would be in int

to force float output

or

```
double dx = x2-x1;
double dy = y2-y1;
return sqrt(dx*dx + dy*dy);
```

Q. check triangle is valid or not.

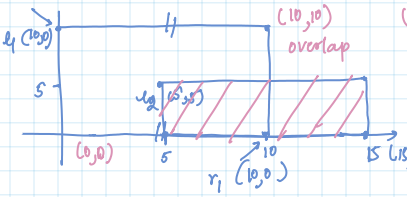
if sum of 2 sides is always greater than 3rd side.

$(a+b) > c$, $(a+c) > b$, $(b+c) > a$

```
if (a+b>c || b+c>a || a+c>b)
        return true;
else
        return false;
```

Q. find if 2 triangles are overlapping.

check if coordinates overlap.



(10,0) & (0,10)
$(x_1,y_1) = (0,0)$
$(x_2,y_2) = (0,10)$
$(x_3,y_3) = (10,10)$
$(x_4,y_4) = (10,0)$

(5,5) & (15,0)
$(x_1,y_1) = (0,5)$
$(x_2,y_2) = (5,5)$
$(x_3,y_3) = (15,5)$
$(x_4,y_4) = (15,0)$

$r_1 = 10,0$ $r_2 = 15,0$
$l_1 = 0,10$ $l_2 = 5,5$

solution:

```
if (l1.x > r2.x || l2.x > r1.x)        (10>15 || 5>10)
        return false;
if (r1.y > l2.y || r2.y > l1.y)        (0>5 || 0>0)
        return false;
return true;
```

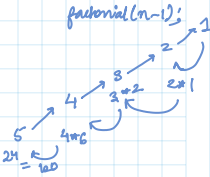
Q. factorial of a no.

Brute force:

```
n=5; fact=0;                O(n)
for (int i=1, i<=5; i++)
        fact *= i;          1*2*3*4*5
```

recursive:

```
factorial(n)
{ if (n==1) return 1;
  return n * factorial(n-1);
}
```



Q. pair cube count

Given: n, count all a & b that satisfy $a^3 + b^3 = n$

(a,b) ≠ (b,a) pair

eg: n=9 → $1^3 + 2^3 = 9$, $2^3 + 1^3 = 9$

ans = 2 pairs

n=28 → $3^3 + 1^3 = 28$, $1^3 + 3^3 = 28$

ans = 2 pairs

brute force: O(n²)

```
for(int i=1; i<=n; i++)
{   for(int j=0; j<=n; j++)
    { if (i*i*i + j*j*j == n)
            count++;
    }
}
```

better: n=28

(n-i) ? cube root of a no.?

(n+i) ? cube root of a no?

check if number of pairs = no of cube root that exist in the range 1 – cbrt(n)

cbrt(28) = 3.03

```
for (int i=1; i<=cbrt(n); i++)
{
    int cb = i*i*i;
    int diff = n-cb;
    int cbrtdiff = cbrt(diff);
    if (cbrtdiff*cbrtdiff*cbrtdiff == diff)
        count++;
}
```

i=1	i=2	i=3
cb = 1	cb = 8	cb = 27
diff = 28-1 = 27	diff = 28-8 = 20	diff = 28-27 = 1
cbrtdiff = 3	cbrtdiff = 2.7	cbrtdiff = 1
if (3*3*3 == diff) count = 1;	X	if (1*1*1 == diff) count = 2

Q. program to find GCD or HCF of 2 no.

36 = 3 x 2 x 2 x 3 = $2^2 \times 3^2$

60 = 3 x 2 x 2 x 5 = $2^2 \times 3 \times 5$

= $2 \times 2 \times 3 = 12$ ← highest common factor

Optimal:

Euclidean

1) O(min(a,b))

```
if (a==0) return b
if (b==0) return a
if (a==b) return a;
if (a>b)
    return gcd(a-b, b);
return gcd(a, b-a);
```

2) O(log(min(a,b))

```
int gcd(int a, int b) {
    return b==0 ? a : gcd(b, a%b);
}
```

or

```
if (a>b)
    if (a%b==0)
        return b
    return gcd(a-b, b)
if b%a==0)
    return a;
return gcd(a, b-a);
```

3) O(log(min(a,b)) + O(1)

```
int gcd(int a, int b)        // inbuilt function
{   return __gcd(a,b);
}
```

Q. perfect number

if it is equal to the sum of its divisors.

n = 36

1 * 36
2 * 18
3 * 12
4 * 9
6 * 6
9 * 4
12 * 3
18 * 2
36 * 1

```
divisor = 1;        // 1 is a divisor always
for (int i=2; i<=sqrt(n); i++)        or i*i<=n
    if (n%i==0)
    {   divisor += i;
        if (n%(n/i)==0)
            divisor += (n/i);
    }
if (divisor == n && n!=1)
        return true;
else
        return false;
```

start from 2

eg: 2 + 18 = 36

O(sqrt(n))
O(1)

Q. lcm of 2 no.

lcm(a,b) * hcf(a,b) = a*b

or lcm(a,b) = $\frac{a*b}{hcf(a,b)}$

O(log(min(a,b))) both

```
int gcd(int a, int b)
    return (b==0) ? a : gcd(b, a%b);

int lcm(int a, int b)
    return (a / gcd(a,b)) * b;
```

Q. Add 2 fractions

Concept → $\frac{a}{b} + \frac{c}{d} = \frac{a \times \left(\frac{lcm(b,d)}{b}\right) + c \times \left(\frac{lcm(b,d)}{d}\right)}{lcm(b,d)}$

Eg: $\frac{2}{4} + \frac{3}{8} = \frac{2 \times \left(\frac{8}{4}\right) + 3 \times \left(\frac{8}{8}\right)}{8}$

$= \frac{2\times2 + 3\times1}{8} = \frac{4+3}{8} = \frac{7}{8}$

code:

```
int gcd (int a, int b)
{   return b==0 ? a : gcd(b%a, a);        O(log(min(a,b)))
}

vector<int> gcd ( vector<int> a, vector<int> b)
{   vector<int> ans;                       ← O(1)
    int denominator = gcd ( a[1], b[1]);
    //Lcm*gcd = a*b;
    denominator = (a[1] * b[1])/denominator;

    int numerator = a[0] * denominator / a[1] +
                    b[0] * denominator / b[1]

    // at this step we have num/deno but it have
    //chances to get reduced further
    int common_factor = gcd(numerator, denominator);

    denominator = denominator / common_factor;
    numerator = numirator / common_factor;

    ans.push_back( numerator);
    ans.push_back( denominator);
}
```

Q. Decimal to binary conversion

Given non negative no, convert given no to equivalent binary repr.

$n = 12 \rightarrow 1100$ as $(2^3)(2^2)(2^1)(2^0)$ = 8 4 2 1 → 1 1 0 0

1) brute: 2)12(6, 12, 0; 2)6(3, 6, 0; 2)3(1, 2, 1; 2)1(0, 0, 1

reverse → 1100

Repeatedly divide by 2 and reverse

```
string decToBinary( int n) {
    string bin = "";
    while (n>0)
    {   int bit = n%2;
        bin.push_back('0' + bit);
        n/=2;
    }
    reverse( bin.begin(), bin.end());
    return bin;
}
```

12 ↓ $\log_2(n)$ 6 ↓ $\log_2(n)$ 3

$T(n) = (\log_2(n))$

'0' is the character literal that converts an integer 0 or 1 into corresponding char 0 or 1.

OR

```
int bit = n%2;
bin.insert( bin.begin(), '0' + bit);
n/=2;          — no reverse
```

$O(\log_2(n))$ for all

2) Bit wise operator

```
int bit = n & 1;
bin.insert( bin.begin(), '0' + bit);
n/=2;
```

12 & 1 → 1100 & 0001 = 0; 6 & 1 → 0110 & 0001 = 0

3 & 1 → 0011 & 0001 = 1; 1 & 1 → 0001 & 0001 = 1

3) inbuilt method

```
string decToBinary (unsigned int n) {
    string s = bitset<64> (n). to_string();
    return s.substr( s.find('1'));     // trim leading zeroes
}
```

Q. find day of the week for a given date

input → d=30, m=8, y=2010

output → 1   30th Aug 2010 was Monday

formulae: not for leap year

day of week = (d + monthCode + yearCode) % 7

steps: 1) calculate month code
2) calculate year code → hard way
3) apply century enhance

[Easier] Zeller's congruence based approach

months have code like

Jan = 6, Feb = 2 (1 for leap), March = 2, April = 5,

May = 0, June = 3, July = 5, August = 1,

Sept = 4, October = 6, Nov = 2, Dec = 4

1) we store all these in t[]
2) do leap year adjustments (y/4 − y/100 + y/400)
3) $y + \frac{y}{4} - \frac{y}{100} + \frac{y}{400} + t[m-1] + d$

code:

```
int dayofweek (int d, int m, int y)
{   static int t[] = {0, 3, 2, 5, 0, 3,
                      5, 1, 4, 6, 2, 4}
    y -= m<3;
    return (y + y/4 - y/100 +
            y/400 + t[m-1] + d)%7;
}

int main() {
    int day = dayofweek (30, 8, 2010);
    cout<<day;
    return 0;
}
```

Q. nth fibonacci using matrix based

```
void multiply (long long a[2][2], long long b[2][2])
{
    long long x = a[0][0]*b[0][0] + a[0][1]*b[1][0];
    long long y = a[0][0]*b[0][1] + a[0][1]*b[1][1];
    long long z = a[1][0]*b[0][0] + a[1][0]*b[1][0];
    long long w = a[1][0]*b[0][1] + a[1][1]*b[1][1];

    a[0][0] = x;  a[0][1] = y;
    a[1][0] = z;  a[1][1] = w;
}
```

[00 01 / 10 11] [00 01 / 10 11] = [00*00 + 01*10, 00*01 + 01*11 / 10*00 + 10*10, 11*01 + 11*11]

O(log(n)) time & O(log(n)) space

Arrays are never copied by value when passed to a function – they become pointers to their first element so modifications are in place.

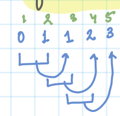
```
void power (long long m[2][2], int n) {
    if (n<=1) return;
    long long base[2][2] = {{1,1},{1,0}};
    power (m, n/2);
    multiply (m, m);      → (1 1 / 1 0)(1 1 / 1 0) = (2 1 / 1 1)
    if (n%2!=0) multiply (m, base);
}                          2%2==0
```

power(m, 2) ← power(m, 1)

```
long long fib (int n) {
    if (n<=1) return n;        taking 0 based index
    //define transpose matrix
    long long m[2][2] = {{1,1},{1,0}};
    power( m, n-1);            // power(m,4)
    return m[0][0];            → as [1 1 / 1 0]^(n-1) = [f(n) f(n-1) / f(n-1) f(n-2)]
}                                 as m[0][0] = f(n)
```

// n=5

Q. nth fibonacci no.



n=2 → fib(2) = 1

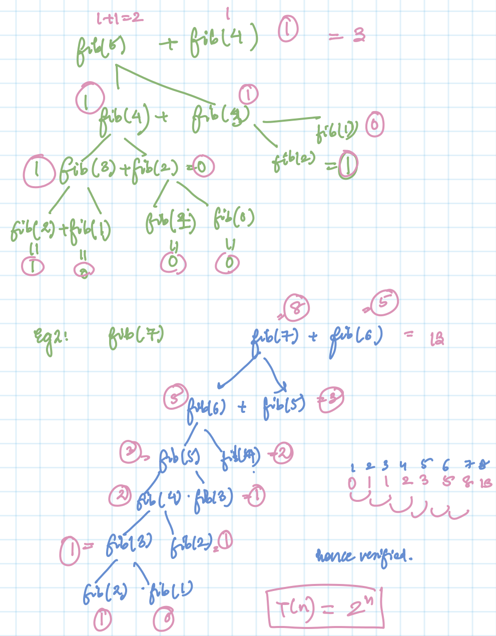
n=5 → 3

\# we know (n=1) = 0, (n=2) = 1 } stored values in series

appears like a recursion

```
int fib (n)
{   if (n==2)
        return 1;
    if (n==1) || (n==0)
        return 0;

    return fib(n) + fib(n-1);
}
```



hence verified.

$T(n) = 2^n$

using memoization – as why should we calculate everything again!

```
vect(int) fib_memoized ( int n, vector<int>& memo)
{   if (n<=1) {return n; }

    if (memo[n] != -1)
        return memo[n];

    memo[n] = fib_memoized (n-1, memo)
              + fib_memoized (n-2, memo);

    return memo[n];
}

int nthfib (int n) {
{   vector<int> memo (n+1, -1);    → create vector for n+1 with all -1
    return fib_memoized ( n, memo);     O(n)
}
```

using loop – dynamic programming like above

```
int fib (int n) {
    if (n==1 || n==0) return 0;
    if (n==2) return 1;
    vector <int> fib (n, -1);
    fib [1] = 0;
    fib [2] = 1;
    for (int i=3; i<n; i++)
    {   fib[i] = fib[i-1] + fib[i-2];
    }
    return fib [n];         O(n)
}
```

we currentno, prev_no1, prev_no2 to reduce O(n) space

0 1 1 2 3 5 8 13

n=5 → fib[3] = 1, fib[4] = 1+1 = 2, fib[5] = 2+1 = 3

n=7 → fib[3] = 1, fib[4] = 2, fib[5] = 3, fib[6] = 5, fib[7] = 8

Use matrix for O(log(n)) + O(log(n))

use transformation matrix – $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$

multiplying by itself gives fibonacci nos.

nth fibonacci no = $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{n-1} = \begin{pmatrix} f(n) & f(n-1) \\ f(n-1) & f(n-2) \end{pmatrix}$

Q. nth term of series : 1, 3, 6, 10, 15, 21 . . .

$n = 3 \rightarrow 6 \quad \frac{n(n+1)}{2} = \frac{3(3+1)}{2} = \frac{3 \times 4}{2} = 6$

$n = 4 \rightarrow 10 \quad \frac{4(4+1)}{2} = 5 \times 2 = 10$

1) brute force O(1) O(1)

```
return (n*(n+1))/2;
```

Q. check if no is a palindrome

```
int m=n;  vector<int> digits;

while (m>0) {
      digits.push_back(m%10);
      m/=10;
}

for (int d: digits)
{   if (d != m%10) return false;
    n/=10;
}
```

Better :

```
bool isPalindrome(int n) {
    int reverse = 0;

    int temp = abs(n);

    while ( temp>0)
    {  reverse = (reverse*10) + (temp%10);
       temp=temp/10;
    }
    return (reverse == abs(n))
}
```

$O(\log_{10} n)$
O(1)

Q. Digital root or repeated digital sum of large int

Brute force → we can use loop and repeatedly divide until n%10 == 0

```
digitalRoot( int n) {

    //sum of digits
    int sum =
    calculateSumofDigits(n);

while (sum/10!=0)
{ sum= calculateSumofDigits(sum);
}
```

```
CalculateSumofDigits(int temp) {
 int sum=0;
 while (temp/10 ==0)
   { int digit = temp%10;
     sum += digits;
   }
 }
 return sum;
```

$O(n^2)$

Optimal :

Sum of digits can be represented as :

$abcd = a \times 10^3 + b \times 10^2 + c \times 10^1 + d \times 10^0$

$\quad = a+b+c+d + (a \times 999 + b \times 99 + c \times 9)$

$abcd = a+b+c+d + 9(a \times 111 + b \times 11 + c \times 1)$

modulo on both sides

$abcd \% 9 = (a+b+c+d)\%9 + 0$

as 9) 9(a×111 + (b11+c)) → 0

```
function SingleDigit()
{
    if (n == 0) return n;
    if (n%9==0) return 9;
    return n%9
}
```

n = 1284

9) 1284 (137 — 9, 38, 27, 64, 63, 1 (remainder)

ans = 1

= 1+2+8+4
= 8+7
= 15 → 1+5 = 6 ... = 10 or 1

Q. Armstrong numbers

given x, determine if given no is Armstrong.

eg: $n = 153 \rightarrow 1 \times 1 \times 1 + 5 \times 5 \times 5 + 3 \times 3 \times 3 = 153$
yes

1) Brute force :

```
int sum=0 ;  int m=n;
vector <int> digits;

while(m>0) {
    digits.push_back( m%10);
    m/=10;
}

int len= digits.size();

for( int d: digits) {
    sum += pow(d, len);
}

return n== sum;
```

T(n) = O(d) → no of digits or $O(\log_{10} n)$ as $d = \log_{10} n + 1$

space = O(d)

Save space by :

```
int len = floor(log10(n)) + 1;   no of digits
```

Optimal!

```
while( m>0 )
{   int digit = m%10;
    sum += pow(digit, len);
    m/= 10
}
```

$O(\log_{10}(n))$
O(1)

Q. Large no divisible by 4

1) do n%4==0 O(1)

1284 → 4x

2) check last 2 digits ÷ 4

```
int temp=n;
last_2_digits =((temp/10)%10)*10
               + (temp%10)
if (last_2_digits %4 ==0)
    return true;
return false;
```

Q. to calculate ^nC_r or combinations

1) use binomial coefficient O(r)

```
for (int i=1; i<=r; i++)
{  sum= sum * (n-r+1)/i; }

return (int)sum;
```

or log (complex)

Q. find kth digit in a^b $O(\log n)$

```
int findkthDigit(int a, int b, int k)
{  int res1= pow(a,b);
   int i= 1;
   while (res1/10 > 0)
   { int digit = res1%10;

     if (i==k) return digit;
     else i++;
     res = res/10;
   } return -1;
}
```

Q. print pascal's triangle

```
    0 1 2 3 4 5 6
0         1
1       1   1
2     1   2   1
3   [ 3   3   3   1
```

i= 1, row=0, col=0

i= 1 → [1, 2], i=1 → [1, 4]
i =1 → [2, 1], i=2 → [3, 3], i=1 [3, 4]

$i=1 \quad 1 * \frac{(1-1)}{1} = 1$

$i=2 \quad 1 * \frac{(1-2)}{2} = 1 * (-1) = -1$

Medium level Questions:

Q.1 Check an int is a perfect square or not

Given +ve integer → return square root or floor of square root if it doesn't exist

1) Brute force $n = 4 \rightarrow 1, 2$ $1^2 \neq 4$, $2^2 = 4$

```
for(int i=1; i*i <= n ; i++)
{     if( pow(i, 2) == n)
           return i;     }
return floor( sqrt(n));
```

O(sqrt(n))

$n = 11 \rightarrow 1^2, 2^2, 3^2, 4^2$ none → sqrt(11) = 3.31 ...
floor(3.31...) = 3

or

```
int res=1;
while ( res*res <= n)
    res++;
return res;
```

2) Better – binary search

```
int high = n, low =1 , res =1;

while( low <= high)
{     int mid = (high + low) /2;

      if (mid*mid <= n)
      {     res=mid
            low= mid+1;
      }
      else {
            high= mid-1;
      }
return res;  }
```

O(log n)

3) Built in

```
int res= sqrt(n)
return res;
```

O(log n)

4) Optimal

if $x = \sqrt{n}$

$x^2 = n$

$\log(x^2) = \log(n)$

$2 \log(x) = \log(n)$

$\log(x) = \frac{\log(n)}{2}$

$x = e^{1/2 * \log(n)}$

```
int res= exp( 0.5 * log(n));

if ( (res+1)* (res+1) <= n)
    res++;

return res;
```

O(1)

n = 11
res = exp(0.5 + log 11) = 3.31... ≈ 3
(res+1)*(res+1) = 4*4 = 16 <= 11 ✗
so return 3;

Q. find no.s from 1-n with exactly 3 divisor.

```
int res =2;   //as 1 & n are always a divisor

for(int j=4; j<=n; j++) {          n+1
  for(int i=2; i*i < j; i++)       n
     if(j %i ==0)
        if ( i== j/i)
           res++;
        else
           res += 2;
     if(res >3) break;
}
```

$O(n^2)$

Better Number must be a square of prime

```
bool isPrime (int n)
{    for (int i=2; i*i<=n; i++)
     {     if (n%i ==0)
               return false   }
     return true;    }
```

```
int no (int n)
{   for (int i=2; i*i<=n; i++)
        if (isPrime(i))
           cout<< i*i << "   "; }
```